UNIVERSIDAD TECNOLÓGICA NACIONAL

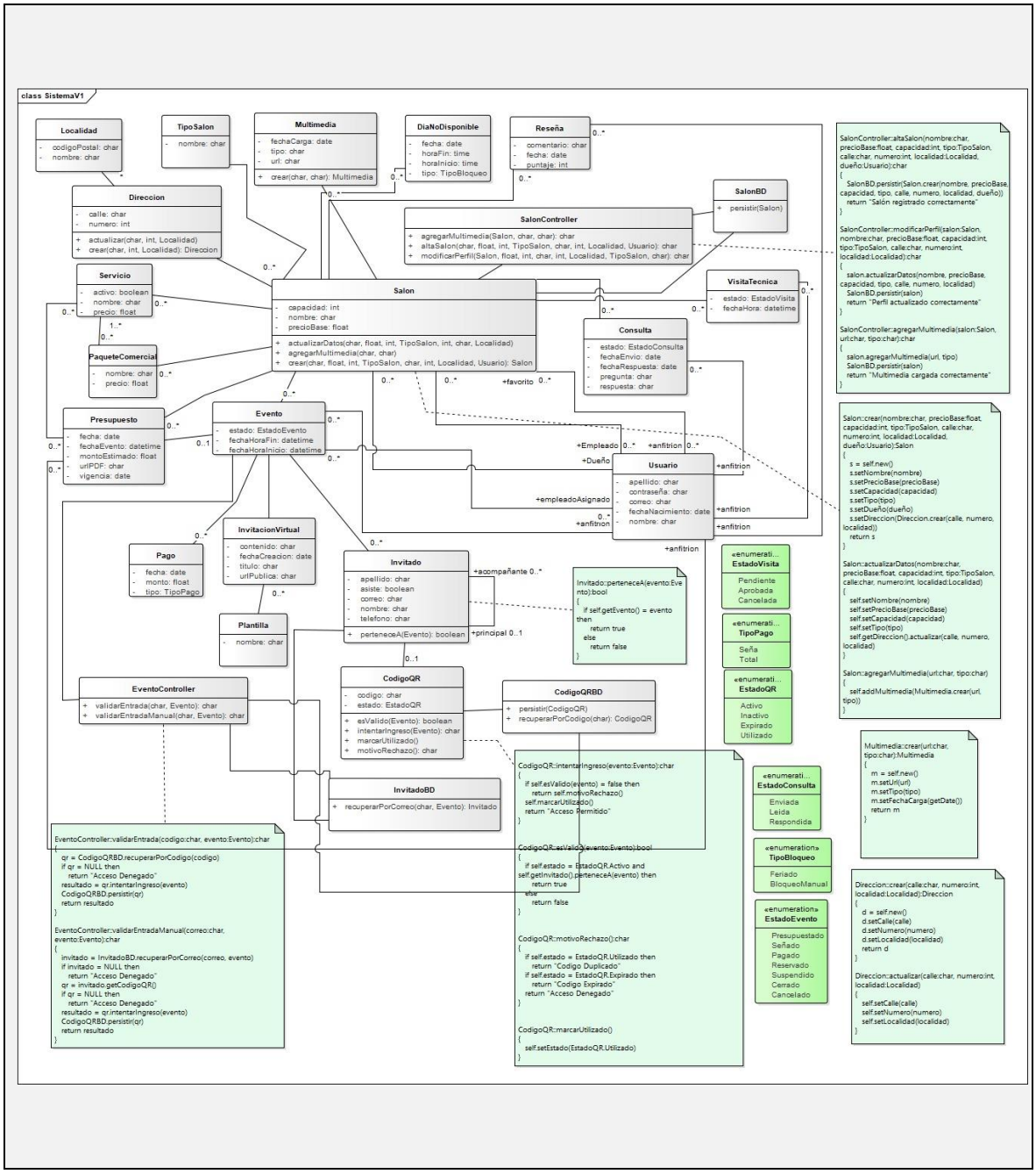
DISEÑO DE SISTEMAS
GRUPO 8

Comisión: 31
Año de cursada: 2026

Integrantes del grupo:

- Mugetti, Emmanuel - Legajo:32534
- Mugetti, Leandro - Legajo: 32535
- Sanchez, Mariano - Legajo: 33079
- Vanucci, Joaquin - Legajo: 33062
- Testa, Daniel - Legajo: 24165

Alcance 3: Eventop


class SistemaV1
Localidad
- codigoPostal: char
- nombre: char
TipoSalon
- nombre: char
Multimedia
- fechaCarga: date
- tipo: char
- url: char
+ crear(char, char): Multimedia
DiaNoDisponible
- fecha: date
- horaFin: time
- horaInicio: time
- tipo: TipoBloqueo
Reseña
- comentario: char
- fecha: date
- puntaje: int
SalonBD
+ persistir(Salon)
Direccion
- calle: char
- numero: int
+ actualizar(char, int, Localidad)
+ crear(char, int, Localidad): Direccion
SalonController
+ agregarMultimedia(Salon, char, char): char
+ altaSalon(char, float, int, TipoSalon, char, int, Localidad, Usuario): char
+ modificarPerfil(Salon, float, int, char, int, Localidad, TipoSalon, char): char
Servicio
- activo: boolean
- nombre: char
- precio: float
Salon
- capacidad: int
- nombre: char
- precioBase: float
+ actualizarDatos(char, float, int, TipoSalon, int, char, Localidad)
+ agregarMultimedia(char, char)
+ crear(char, float, int, TipoSalon, char, int, Localidad, Usuario): Salon
VisitaTecnica
- estado: EstadoVisita
- fechaHora: datetime
PaqueteComercial
- nombre: char
- precio: float
Consulta
- estado: EstadoConsulta
- fechaEnvio: date
- fechaRespuesta: date
- pregunta: char
- respuesta: char
Presupuesto
- fecha: date
- fechaEvento: datetime
- montoEstimado: float
- urlPDF: char
- vigencia: date
Evento
- estado: EstadoEvento
- fechaHoraFin: datetime
- fechaHoraInicio: datetime
Usuario
- apellido: char
- contraseña: char
- correo: char
- fechaNacimiento: date
- nombre: char
InvitacionVirtual
- contenido: char
- fechaCreacion: date
- titulo: char
- urlPublica: char
Pago
- fecha: date
- monto: float
- tipo: TipoPago
Invitado
- apellido: char
- asiste: boolean
- correo: char
- nombre: char
- telefono: char
+ perteneceA(Evento): boolean
Plantilla
- nombre: char
CodigoQR
- codigo: char
- estado: EstadoQR
+ esValido(Evento): boolean
+ intentarIngreso(Evento): char
+ marcarUtilizado()
+ motivoRechazo(): char
CodigoQRBD
+ persistir(CodigoQR)
+ recuperarPorCodigo(char): CodigoQR
EventoController
+ validarEntrada(char, Evento): char
+ validarEntradaManual(char, Evento): char
InvitadoBD
+ recuperarPorCorreo(char, Evento): Invitado
«enumerati... EstadoVisita
Pendiente
Aprobada
Cancelada
«enumerati... TipoPago
Seña
Total
«enumerati... EstadoQR
Activo
Inactivo
Expirado
Utilizado
«enumerati... EstadoConsulta
Enviada
Leida
Respondida
«enumeration» TipoBloqueo
Feriado
BloqueoManual
«enumeration» EstadoEvento
Presupuestado
Señado
Pagado
Reservado
Suspendido
Cerrado
Cancelado
SalonController::altaSalon(nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad, dueño:Usuario):char
{
SalonBD.persistir(Salon.crear(nombre, precioBase, capacidad, tipo, calle, numero, localidad, dueño))
return "Salón registrado correctamente"
}
SalonController::modificarPerfil(salon:Salon, nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad):char
{
salon.actualizarDatos(nombre, precioBase, capacidad, tipo, calle, numero, localidad)
SalonBD.persistir(salon)
return "Perfil actualizado correctamente"
}
SalonController::agregarMultimedia(salon:Salon, url:char, tipo:char):char
{
salon.agregarMultimedia(url, tipo)
SalonBD.persistir(salon)
return "Multimedia cargada correctamente"
}
Salon::crear(nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad, dueño:Usuario):Salon
{
s = self.new()
s.setNombre(nombre)
s.setPrecioBase(precioBase)
s.setCapacidad(capacidad)
s.setTipo(tipo)
s.setDueño(dueño)
s.setDireccion(Direccion.crear(calle, numero, localidad))
return s
}
Salon::actualizarDatos(nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad)
{
self.setNombre(nombre)
self.setPrecioBase(precioBase)
self.setCapacidad(capacidad)
self.setTipo(tipo)
self.getDireccion().actualizar(calle, numero, localidad)
}
Salon::agregarMultimedia(url:char, tipo:char)
{
self.addMultimedia(Multimedia.crear(url, tipo))
}
Multimedia::crear(url:char, tipo:char):Multimedia
{
m = self.new()
m.setUrl(url)
m.setTipo(tipo)
m.setFechaCarga(getDate())
return m
}
Direccion::crear(calle:char, numero:int, localidad:Localidad):Direccion
{
d = self.new()
d.setCalle(calle)
d.setNumero(numero)
d.setLocalidad(localidad)
return d
}
Direccion::actualizar(calle:char, numero:int, localidad:Localidad)
{
self.setCalle(calle)
self.setNumero(numero)
self.setLocalidad(localidad)
}
Invitado::perteneceA(evento:Evento):bool
{
if self.getEvento() = evento then
return true
else
return false
}
CodigoQR::intentarIngreso(evento:Evento):char
{
if self.esValido(evento) = false then
return self.motivoRechazo()
self.marcarUtilizado()
return "Acceso Permitido"
}
CodigoQR::esValido(evento:Evento):bool
{
if self.estado = EstadoQR.Activo and self.getInvitado().perteneceA(evento) then
return true
else
return false
}
CodigoQR::motivoRechazo():char
{
if self.estado = EstadoQR.Utilizado then
return "Codigo Duplicado"
if self.estado = EstadoQR.Expirado then
return "Codigo Expirado"
return "Acceso Denegado"
}
CodigoQR::marcarUtilizado()
{
self.setEstado(EstadoQR.Utilizado)
}
EventoController::validarEntrada(codigo:char, evento:Evento):char
{
qr = CodigoQRBD.recuperarPorCodigo(codigo)
if qr = NULL then
return "Acceso Denegado"
resultado = qr.intentarIngreso(evento)
CodigoQRBD.persistir(qr)
return resultado
}
EventoController::validarEntradaManual(correo:char, evento:Evento):char
{
invitado = InvitadoBD.recuperarPorCorreo(correo, evento)
if invitado = NULL then
return "Acceso Denegado"
qr = invitado.getCodigoQR()
if qr = NULL then
return "Acceso Denegado"
resultado = qr.intentarIngreso(evento)
CodigoQRBD.persistir(qr)
return resultado
}

RF-001 — Gestión de Perfil del Salón

- **Descripción:** Registrar y modificar datos del perfil: tipo, precio base/hora, ubicación, capacidad, disponibilidad y multimedia.
- **Escenario de prueba:** Modificar el precio base y la capacidad del salón, verificar que los cambios se guarden y actualicen inmediatamente.
- **Finalidad:** Permitir al salón exponer su oferta comercial parametrizada en la plataforma. Objetivo EvenTop: Registro de salones, Encontrar salones por disponibilidad.

RF-214 — Control de Acceso

- **Descripción:** Validar la entrada de invitados en tiempo real (manual o vía escaneo QR).
- **Escenario de prueba:** Escanear un código QR válido en la entrada del salón y validar que la pantalla devuelva un mensaje verde de "Acceso Permitido".
- **Finalidad:** Profesionalizar el ingreso al evento, erradicando los fraudes y las largas filas en la puerta.

----------RF 001----------

```
SalonController::altaSalon(nombre:char, precioBase:float,
capacidad:int, tipo:TipoSalon, calle:char, numero:int,
localidad:Localidad, dueño:Usuario):char
{
   SalonBD.persistir(Salon.crear(nombre, precioBase, capacidad,
tipo, calle, numero, localidad, dueño))
   return "Salón registrado correctamente"
}

SalonController::modificarPerfil(salon:Salon, nombre:char,
precioBase:float, capacidad:int, tipo:TipoSalon, calle:char,
numero:int, localidad:Localidad):char
{
   salon.actualizarDatos(nombre, precioBase, capacidad, tipo, calle,
numero, localidad)
   SalonBD.persistir(salon)
   return "Perfil actualizado correctamente"
}

SalonController::agregarMultimedia(salon:Salon, url:char,
tipo:char):char
{
   salon.agregarMultimedia(url, tipo)
   SalonBD.persistir(salon)
   return "Multimedia cargada correctamente"
}



Salon::crear(nombre:char, precioBase:float, capacidad:int,
tipo:TipoSalon, calle:char, numero:int, localidad:Localidad,
dueño:Usuario):Salon
{
```

```
   s = self.new()
   s.setNombre(nombre)
   s.setPrecioBase(precioBase)
   s.setCapacidad(capacidad)
   s.setTipo(tipo)
   s.setDueño(dueño)
   s.setDireccion(Direccion.crear(calle, numero, localidad))
   return s
}

Salon::actualizarDatos(nombre:char, precioBase:float, capacidad:int,
tipo:TipoSalon, calle:char, numero:int, localidad:Localidad)
{
   self.setNombre(nombre)
   self.setPrecioBase(precioBase)
   self.setCapacidad(capacidad)
   self.setTipo(tipo)
   self.getDireccion().actualizar(calle, numero, localidad)
}

Salon::agregarMultimedia(url:char, tipo:char)
{
   self.addMultimedia(Multimedia.crear(url, tipo))
}

Multimedia::crear(url:char, tipo:char):Multimedia
{
   m = self.new()
   m.setUrl(url)
   m.setTipo(tipo)
   m.setFechaCarga(getDate())
   return m
}

Direccion::crear(calle:char, numero:int,
localidad:Localidad):Direccion
{
   d = self.new()
   d.setCalle(calle)
   d.setNumero(numero)
   d.setLocalidad(localidad)
   return d
}

Direccion::actualizar(calle:char, numero:int, localidad:Localidad)
{
   self.setCalle(calle)
   self.setNumero(numero)
   self.setLocalidad(localidad)
}
```

----------RF214----------

```
EventoController::validarEntrada(codigo:char, evento:Evento):char
{
```

```
   qr = CodigoQRBD.recuperarPorCodigo(codigo)
   if qr = NULL then
      return "Acceso Denegado"
   resultado = qr.intentarIngreso(evento)
   CodigoQRBD.persistir(qr)
   return resultado
}

EventoController::validarEntradaManual(correo:char,
evento:Evento):char
{
   invitado = InvitadoBD.recuperarPorCorreo(correo, evento)
   if invitado = NULL then
      return "Acceso Denegado"
   qr = invitado.getCodigoQR()
   if qr = NULL then
      return "Acceso Denegado"
   resultado = qr.intentarIngreso(evento)
   CodigoQRBD.persistir(qr)
   return resultado
}

CodigoQR::intentarIngreso(evento:Evento):char
{
   if self.esValido(evento) = false then
      return self.motivoRechazo()
   self.marcarUtilizado()
   return "Acceso Permitido"
}


CodigoQR::esValido(evento:Evento):bool
{
   if self.estado = EstadoQR.Activo and
self.getInvitado().perteneceA(evento) then
      return true
   else
      return false
}


CodigoQR::motivoRechazo():char
{
   if self.estado = EstadoQR.Utilizado then
      return "Codigo Duplicado"
   if self.estado = EstadoQR.Expirado then
      return "Codigo Expirado"
   return "Acceso Denegado"
}


CodigoQR::marcarUtilizado()
{
   self.setEstado(EstadoQR.Utilizado)
```

```
}

Invitado::perteneceA(evento:Evento):bool
{
   if self.getEvento() = evento then
      return true
   else
      return false
}
```